Sorting

1.

Renowned Progressive Rock band 'Porcupine Tree' released an album called 'Closure/Continuation' after about 13 years. Now as a Progressive Rock Music fan you are going to listen to the tracks of the album but in the order of their Youtube views (highest one at first, lowest one at last) instead of the order of the album tracklist.

You have chosen an Algorithm to order them as per your preference. If multiple tracks have the same views, you are going to listen to any of them the Algorithm puts first in the list after ordering. This algorithm also solves your concern about the issues regarding space that your device is facing. Porcupine Tree made the fans wait for 13 years for a new album and so you think you have the patience to wait as much time as the Algorithm may take to order the tracks.

The following table contains the list of the tracks and their Youtube views (in thousand) :

Track	Harridan	Of The New Day	Rats Return	Dignity	Herd Culling	Walk the Plank	Chimera's Wreck	Population Three
Views	15	8	11	112	33	39	88	41

a. CO2 **Specify** the name of the algorithm you have chosen and **simulate** the Algorithm to order the tracks of the album as per your preference. Show your workings in detail. **7**

b. CO5 Suppose you want to run the algorithm again on the ordered list. **Determine** the run-time complexity of the algorithm in this scenario. **3**

2.

While sorting a list of 10 integers in ascending order using Quick Sort, after the first partition (using the first element as a pivot), the list looks like this : [31, 19, 3, 17, 23, 37, 59, 61, 71, 43]. Which element was the pivot before partitioning?

Explain your answer in brief.

3.

Jack loves to play with integers. He created a list of n integers where the even indices hold numbers in decreasing order and the odd indices hold numbers in increasing order. For example, this is a list of n=8 integers Jack made.

Index	0	1	2	3	4	5	6	7
Number	23	2	19	3	7	11	5	13

[Explanation:
The indices 1, 3, 5, and 7 have numbers 2, 3, 11, and 13 in increasing order.
The indices 0, 2, 4, and 6 have numbers 23, 19, 7 and 5 in decreasing order.]

You sorted the the list in quadratic time [O(n^2)]. To your utter surprise, Jack replies, "It could be sorted in linear time".

1) ***[CO2]*** **Show** the steps how you sorted the list using a suitable algorithm. Mention the name of the algorithm. — 3
2) ***[CO2]*** **Describe** how Jack sorted the list in linear time. Show the steps too.— 4
3) ***[CO3]*** Jack wants to add the number 15 to your sorted list from (1). He wants you to come up with an efficient idea about how he is going to find the accurate index position for 15 in the list.
   Describe how you are going to assist him in this. **Show** the steps too. — 3

4.

While sorting a list of 10 integers in ascending order using Quick Sort, after the first partition (using the first element as a pivot), the list looks like this : [13, 11, 19, 7, 23, 37, 29, 53, 59, 41]. Which element was the pivot before partitioning?
Explain your answer in brief.

5.

Simulate the Quick Sort algorithm by showing the steps on the following array, always choosing the last element as the pivot element. You can partition the array at each step choosing any suitable algorithm, *but you do not need to show the partitioning steps*.

51	70	28	12	75	62	88	43

6.

Consider an array containing **N** unique values where for some index **i**, the values are in increasing order from index **0** to (**i-1**), and then again from **i** to **(N-1)**. An example array is given below.
Here **i=3**, it means the values are in increasing order from index 0 to 2, and then again from 3 to 7.

index	0	1	2	3	4	5	6	7
value	9	12	15	2	4	5	7	8

Given the value of **i**, propose an algorithm to search a *key_value* in the array. Complexity of your algorithm must be less than **O(N)**.

a) Present your solution idea as a code/ pseudocode/ flowchart/ step-by-step instructions/ logical explanations in short.
b) Write the time complexity of your presented solution.
c) Show a simulation of the Merge Sort algorithm to organize the whole array in increasing order.

7.

For different tasks, we often need to search for things. And most of the time, we sort the dataset before performing search operations. Once, you found the following list of numbers.

[2, 5, 1.2, 6.7, 1.7, 9.3, 2.2, 7.7, 0, –4, –5.1, 2, 5, 5.2]

a. To search an element in an unsorted array, we need O(N) time using linear search. Binary search works in O(logN) time but the array needs to be sorted beforehand which takes at least O(NlogN) time in general. Why would you then sort the array first and then perform binary search instead of just performing linear search?
b. Can you modify count sort so that it may work with negative integers as well?
c. Can you modify count sort so that it may work with the given list?
d. Say, you are working in a system where you need to sort a very big dataset. The memory available to you can barely accommodate the data. You have two options – merge sort & quick sort. Which one should you choose? Why?
e. Construct an array where quick sort fails to work in O(NlogN) time.

8.

a. **CO1** You are designing a computerized logistics system for a warehouse that handles thousands of packages daily. Each package is assigned a unique package ID and a delivery deadline (in terms of 'remaining hours' from the current time) and the packages are initially sorted according to their package_id. The warehouse management system needs to sort the packages by their delivery deadlines in ascending order to prioritize the earliest deliveries.

But there are some conditions to develop this system. The system does not have any issues related to space/memory but it needs the list to be sorted with time complexity $O(nlogn)$ where n is the number of packages to process. Moreover, if any two packages have the same delivery deadline, the package with smaller package_id comes before i.e the algorithm must be stable.

Write the name of the appropriate algorithm for this case and show the solution with pseudocode/programmable code/step-by-step logical instructions.

b. [Continues on the previous question…] After some days, the warehouse noticed a massive increase in daily orders and the system was facing memory issues to hold these massive orders. So they consulted you again. You suggested to them an algorithm which does not require any additional memory. But you also warned them that it can degrade to the time complexity of $O(n^2)$ in some cases. However, they insist the time complexity must remain $O(nlogn)$ in all cases. So you proposed a modification to your algorithm for question (a) without increasing the time complexity.

CO1 i. **Name** the algorithm you suggested them and explain the case where the algorithm might take $O(n^2)$ time.

CO2 ii. What solution did you come up with to ensure $O(nlogn)$? **Explain** where it differs from the solution of question (a).

9.

Write the worst case time complexity of quick sort? **Illustrate** an array where the worst case of quick sort occurs if the last element is chosen as pivot.

10.

b. **CO2** In case of sorting an array in ascending order, *Bubble Sort* extends a sorted subarray at the rightmost side and, *Insertion Sort* extends a sorted subarray at the leftmost side. Your friend, Jimmy was told to find the first five largest and smallest numbers from a list of N distinct integers ($N > 10$). To solve the task, he modified the Bubble sort and Insertion sort algorithm for only 5 iterations and used the rightmost and leftmost 5 numbers as the 5 largest and smallest numbers respectively. Do you support his strategy? **Explain** with logical reasons.

c. **CO2** While sorting a list of 10 integers in ascending order using Quick Sort, after the first partition (using the first element as a pivot), the list looks like this : [13, 11, 19, 7, 23, 37, 29, 53, 59, 41]. Which element was the pivot before partitioning?
Explain your answer in brief.

d. **CO2** Consider the scenario in 2(c). **Show** how the list will look like after partitioning using **13** as the pivot.

11.

c. **CO1** Suppose we implement Quicksort using a strategy where we choose the **pivot as the median of the first, last, and middle elements of a subarray**. To be specific, the pivot is selected as follows: **03**

$$\text{pivot} = \text{median}(A[\text{st}], A[\text{end}], A[\text{mid}]) \quad \text{where} \quad \text{mid} = \left\lfloor \frac{\text{st} + \text{end}}{2} \right\rfloor$$

For example, consider the array A = [6, 2, 9, 1, 7, 3, 8].
In the initial Quicksort call, start = 0 gives A[0] = 6, end = 6 gives A[6] = 8, and mid index $= \frac{(0+6)}{2} = 3$ gives A[3] = 1. Among these three values—1, 6, and 8—the sorted order is (1, 6, 8), so the median, and thus the chosen pivot, is 6.

What is the worst-case input for this pivot selection strategy in Quicksort? **Generate** such a worst-case scenario with a 7-element array containing the integers 1 through 7, each used exactly once. Briefly explain why it leads to worst-case behavior.

Now consider a factory where **n** humanoid robots work. You are to sort the robots based on some criteria. Answer Q2(d) – (e) based on this.

d. **CO1** Each robot can form connections with some of the other robots. That means each robot can form at most (n-1) connections. Which algorithm will be the fastest to sort the robots based on the number of connections they formed? **Write** the name and time complexity of the algorithm. **02**

e. **CO1** Suppose, in the factory, there exists a system that can find the tallest robot in O(1) time. That means you are given a function, $find_tallest(A)$, which returns the tallest robot from a list called **A**, in constant time. **03**

Considering this information, design an algorithm to sort the robots in descending order of their heights. **The time complexity of your algorithm must not exceed O(n).**

i. **Present** your algorithm using pseudocode/ executable code/ step-by-step logical instructions.
ii. **Write** the time complexity of your algorithm.

12.

a **CO1**	You are developing an e-commerce platform that collects product ratings from users (out of five stars). For analytics, the system needs to sort millions of these ratings quickly every day. Memory is not an issue, but comparison operations are expensive. Which algorithm is the most suitable for the above scenario? **Explain** your choice. **Write** the time complexity of your preferred algorithm.	**02**
b **CO3**	Suppose your friend wants to implement a variant of Merge Sort called Ternary Merge Sort, where instead of dividing the array into two halves, **he divides it into three equal (or nearly equal) parts** and recursively sort each part, and then merge the three sorted subarrays into a single sorted array. It is guaranteed that the input array will have a size of at least 3. Now, he asks your help to **implement the 'dividing step' of the algorithm**. Present your idea with pseudocode/programmable-code/step-by-step logical instructions.	**02**
c **CO2**	Does dividing into three subarrays make the algorithm asymptotically faster? **Explain** why or why not.	**01**

13.

a **CO1**	You are working on a grading system for a national standardized exam where students are marked out of 100. You need to sort millions of student scores as efficiently as possible. Speed is critical, and memory is not a major constraint. Comparisons are costly. Which algorithm is the most suitable for the above scenario? **Explain** your choice. **Write** the time complexity of your preferred algorithm.	**02**
b **CO3**	Suppose your friend wants to implement a variant of Merge Sort called Ternary Merge Sort, where instead of dividing the array into two halves, **he divides it into three equal (or nearly equal) parts** and recursively sort each part, and then merge the three sorted subarrays into a single sorted array. It is guaranteed that the input array will have a size of at least 3. Now, he asks your help to **implement the 'dividing step' of the algorithm**. Present your idea with pseudocode/programmable-code/step-by-step logical instructions.	**02**
c **CO2**	Does dividing into three subarrays make the algorithm asymptotically faster? **Explain** why or why not.	**01**

14.

2	**a.** **CO1**	You are given an array A consisting of N integers. Some of the numbers are positive, and some are negative. For example $A: \{15, -13, 3, -4, 5, -6\}$. You have to organize the array in a way that all the negative numbers come to the front and all positive numbers are pushed to the back. For example, the array A from above becomes $Org_A: \{-6, -4, -13, 15, 3, 5\}$. *Note that numbers don't need to be sorted here.* Propose a linear-time $[O(N)]$ algorithm with constant $[O(1)]$ space complexity. **Present** your algorithm as pseudocode or executable code, or step-by-step logical instructions. If unable, show a simulation of any algorithm (no limit on the time or space complexity) on the given example array A for 40% marks.	**05**
	b. **CO1**	After solving Q2(a), you want to find the index of the leftmost positive number in the organized array Org_A. Present a modification of Binary Search to achieve this. **Write** either pseudocode or executable code.	**03**

A

	c. **CO2**	Mention the name and time complexity of the most efficient sorting algorithm among Bubble, Insertion, Selection, Merge, Quick, and Count for each of the following input scenarios, consisting of an array of N integers. For each case, you do not have any information other than the given one. Case #1: The largest number can be in any index, but all other numbers are already sorted. Case #2: The smallest number is 0, and the largest is 2^N. Case #3: The array contains all the integers from 1 to N. Case #4: The numbers are in the reverse order of how we want them to be sorted.	**02**